

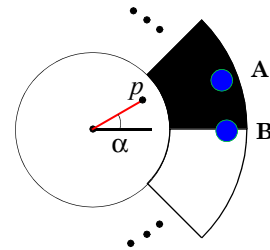
QUADRATURE ENCODER

ABSTRACT: This project involves using the two sensor outputs from quadrature shaft encoder to construct an estimate of the wheel angle over time. The shaft encoder contains a simple quadrature encoder using two optimally-placed sensors that capture the light intensity change on a black/white alternating pattern on the inside of the wheel. Given the sensor signals, the goal is to determine the tic count that relates to the rotational position of the wheel as well as the angle position of the wheel. The issue of the sample time and its effect on the ability of the shaft encoder to accurately estimate the wheel angle versus time is explored.

I. INTRODUCTION

To provide a setting for calibrating the scale of movement, assume that the wheel has a diameter of 4" and that the quadrature encoder has N_b black patches and the same number of white patches which provides a total of $2N_b$ color transitions in one revolution of the wheel.

The basic quadrature encoder is described in the lecture. It consists of a simple black-white pattern with two sensors which we refer to as the A and B sensors. A simplified view of this is shown at the right from the viewpoint of the inside of the wheel. These sensors are staggered by half the width of a single patch so that forward and reverse directions can be detected.



The signals that are detected from the sensors will detect a high voltage for black and a low voltage for white. For the purpose of interpreting the rotational direction correctly, assume that the sensors are fixed in space as shown in the diagram. If the wheel is rotating counterclockwise, the white patch will move up and the signal coming from sensor A will transition from black to white *after* the sensor B transitions from black to white. A *positive* angle change occurs when the wheel is moving *counterclockwise*. Furthermore, the nominal voltage range for the sensor output is $[0, 5]$ volts, and the sensor signals may exhibit random gaussian noise.

II. MATLAB

The sensor output signals are provided by the following instructor-provided matlab function

```
[A,B] = encoder_signals(t,alpha,Nb,LvL)
```

where $\mathbf{t}$ is the time vector, $\mathbf{alpha}$ is the angular position of the wheel in units of *radians* (paired with $\mathbf{t}$), $\mathbf{Nb}$ is the number of black patches on the encoder wheel pattern, and $\mathbf{LvL}$ is a noise level in the range $[0,10]$ where 0 represents no noise and 10 represents maximum noise. The outputs $\mathbf{A}$ and $\mathbf{B}$ are the sensor signals and will have the same length as input vector $\mathbf{t}$. The vectors $\mathbf{t}$ and $\mathbf{alpha}$ must be the same length and define the angular position of the wheel at each sample time.

To clarify the definition of $\mathbf{alpha}$, imagine a fixed point p on the wheel as shown in the diagram above. The line that connects p to the origin is shown in red. That line has some angle α relative to the reference line shown in black. If the wheel is rotating *counterclockwise*, then angle α is increasing. We assume that the reference line, shown in black, does not move with the wheel but point p does move with the wheel.

It is important to note that function `encoder_signals.m` simulates the finite sensor area in the sense that when a black-white boundary passes over the detector, the detector does not change instantly but rather changes linearly assuming that the sensor angular width is $1/20$ that of the angular width of a black patch. This gives a more practical sensor signal.

The user must define a desired rotation of the wheel as a function of time by defining the time vector $\mathbf{t}$ and corresponding vector $\mathbf{alpha}$. An instructor-provided function will generate 4 different signals for use.

Any other function can also be generated. The form of this function is

```
[t,alpha] = alpha_gen(index,T,tspan);
```

where **index** is an integer value between 1 and 4, **T** is the sample time for the output time vector **t**, and **tspan** is the duration of the output time vector. The generated alpha function are defined as follows:

```
index = 1 , spin one revolution counter clockwise (linear movement)
index = 2 , spin one revolution CCW and one rev CW (linear movement)
index = 3 , spin one revolution CCW and one rev CW (cosine movement)
index = 4 , spin one revolution CCW (exponential movement)
```

These signals provide enough flexibility to easily and quickly explore concepts related to the movement of a wheel and the shaft encoder signals that result.

III. REQUIREMENTS

Write a matlab function with the form

```
function [cV,eV,aV] = tic_counter(A,B,Nb);
```

where **A** and **B** are the vectors that define the sensor signals $A(nT)$ and $B(nT)$, where **Nb** is the number of black patches, where output **cV** is the shaft encoder total tic count vector, **eV** is a vector of the same length with values of 1 where state transition errors occur and 0 where there are no state transition errors, and where **aV** is a vector that estimates the wheel angle signal **alpha** using **cV**. A state transition is considered in error if the transition is 2 state changes which gives rise to an ambiguous wheel movement direction.

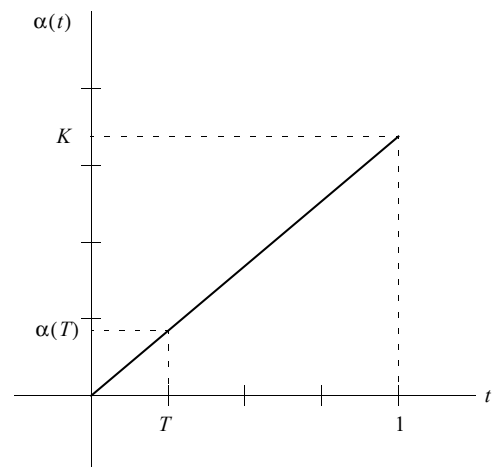
A well-written computer program function, such as this, should check the input variables for consistency. For example, the function should check to make sure that the input vectors **A** and **B** are the same length. If they are not the same length, the function should return empty variables for the 3 output vectors. A vector is set to empty as follows

```
cV = []; eV = []; aV = [];
```

Also, a well-written computer program using any language should be properly documented. A properly documented program includes a documentation header like that shown in the function template found in the matlab setup structure, as well as line comments that describe the broad goal of sections of code. Line comments should not put the syntax into english form, rather it should explain in conceptual terms what the subsequent lines of code are achieving.

IV. STUDY

As a part of this project, you are to investigate the relationship between the number of states per revolution N_s , the sample time T , and the derivative of $\alpha(t)$. This relationship can be thought through with the simple example of a wheel rotating at constant speed as shown in the graph at the right. Suppose the wheel goes through angle K radians in 1 second. Then the spin rate is K rad/sec which is the slope of the line. Suppose the shaft encoder output signal state is sampled at time 0 and again at time T . During this time interval, the shaft will go through some angle. This angle cannot be so large that the state transition goes from 0 to 2, or 1 to 3, both of which lead to an indeterminate tic change. To operate properly, the shaft encoder state change can only change by 1 value for each sample time. So the question is, what is the smallest angle that can cause the state to change from 0 to 2 or from 1 to 3? This



angle is related to N_s . What is the formula that relates N_s , T and K ? This formula should be an inequality where one variable is chosen relative to the other two. This can be expressed in many ways. You just need to find one way to express it.

As part of this study, you are to consider the case of a quadrature encoder with two sensors A and B as described above and with N_b black patches on the encoder wheel pattern. For this case, determine how N_s is related to N_b . Use the value of N_b and T specifically assigned to you as given on the course website to find the fastest spin rate K that the shaft encoder can track.

Using your values of N_b and T , demonstrate your answer by selecting two values of spin rate K , one that the encoder can track and one that it cannot track. Generate two plots, one for each value of K . Each should show two plots, the actual plot of $\alpha(t)$ in blue, and an estimate of $\alpha(t)$ in blue. The estimate should use the instructor-provided functions, your `tic_counter.m` function, as well as appropriate scaling to estimate $\alpha(t)$. See the matlab tutorial for help on how to do this. Put this code in a single script called `mystudy.m` and upload it to your online library in directory `project1`.

V. REPORT

The report format should have the following sections:

1. Title page (title, project #, name, class, semester, date, etc.) and should be a separate page
2. INTRODUCTION - a brief (1 paragraph) description of what this project is about and what is found in this report.
3. RESULTS - This should contain a description of the programs written and all results. The following specific subsections are to be included:
 - a. A brief (1 paragraph) description of the code written for the matlab function `tic_encoder.m`. This section should describe in detail how this function works, including the state definitions and the nature of the state counting procedure.
 - b. A brief (1 page) description of the code written for the matlab script `mystudy.m`. Include image files for the figures that are produced by this function and describe what they mean. This section should also clearly state the formula that relates N_s , T and K and describe the figure results that relate to this equation.
4. CONCLUSIONS - a brief (1 paragraph) description of conclusions drawn.

The report should be no longer than 2 pages of text not including figures and data tables.

VI. DELIVERABLES

All of these should be uploaded to your online library in the directory `project1` by the due date.

1. The matlab function: `tic_encoder.m`
2. The matlab script: `mystudy.m`
3. The matlab report: `report.pdf`

VII. GRADING:

This project will be graded according to how well your matlab code works and the content and completeness of the report. The instructor needs to be able to independently verify your results from the code you submit so it needs to run without error. If the any matlab function or script does not have the proper form for the output vector or causes a run-time error, your project grade will be reduced significantly. If your program takes an inordinate amount of time to run, that will also incur a significant reduction in grade. Work to make your matlab code as computationally efficient as possible.